Retrace API and Event Contracts Specification

Version: 1.0

Date: 6 November 2026

Status: Implementation-ready baseline

Applies to: Core, Ingestion, AI, Notifications, Gateway, and shared contract packages

1. Purpose and authority

This document defines the transport-level contracts that let Retrace services, the web application, and coding agents work together without guessing. It turns the established product, system, database, and AI-pipeline decisions into API and event rules. It does not redefine product scope, persistence ownership, provider behavior, numeric thresholds, or source-specific authority rules.

1.1 Decision status labels

Label	Meaning
Confirmed	An explicit project decision, including the 30 API and event decisions approved in the project conversation. Implement it as written.
Established source decision	A decision already made in the PRD, System Design, Database Design, Project Context, or AI Pipeline Specification.
Standard convention	A compatibility-oriented implementation convention adopted here. It must not introduce product behavior, schema columns, routes, ownership, provider rules, or numeric limits.
Open implementation item	Required detail intentionally left unresolved. Do not guess; resolve it in the central contract package before implementation relies on it.

1.2 Authority and reconciliation order

1. Confirmed decisions in this document take precedence over older documents where they address the same contract question.
2. The AI Pipeline Specification governs evidence-grounded AI behavior. Its source-specific authority rule is already settled: apply source-specific authority where appropriate and preserve contradictions. This contract does not reopen that decision.
3. The System Design governs service boundaries and the endpoint inventory.
4. The Database Design governs existing persistence ownership and documented columns until a contract decision requires a targeted migration.
5. The PRD governs product intent and user-facing workflow.

1.3 Explicit reconciliations

Topic	Earlier material	Resolved contract	Implementation consequence
API version path	System Design used `/api/v1/orgs/{org_id}/`; auth and organization creation were exceptions.	**Confirmed:** public APIs use `/api/v1`; tenant-scoped endpoints use `/api/v1/orgs/{org_id}/`.	Preserve the established auth and organization-creation exceptions under `/api/v1`; do not create an unversioned public API.
Postmortem versions	Database Design leaves a manual-version limit open; PRD names automatic snapshots.	**Confirmed:** immutable rolling queue of 15 named versions; explicit save and AI regeneration create versions; autosave is only a temporary working draft.	Add/adjust retention behavior without changing immutable history in place. A version dropped from the working queue must be handled under the approved retention/deletion policy; the precise archival mechanism is open.
Deletion	Database Design says tenant business data is hard-deleted, while the contract decision says important records are soft-deleted by default.	**Confirmed:** public-resource deletion defaults to soft deletion for important records. Tenant and incident deletion retain their established destructive cleanup behavior.	Add the necessary lifecycle fields/migrations only after their exact names and affected tables are approved in the central contract package. Do not imply existing columns already implement soft deletion.
Evidence timestamp fields	Earlier database schema lists only normalized `occurred_at` for evidence/timeline rows.	**Confirmed:** normalized evidence exposes five timestamp fields in section 10.	Reconcile persistence in a migration; no field may fabricate precision or timezone certainty.
System Design version label	Project Context calls the system design v0.4; the uploaded System Design PDF is labeled v0.3.	The uploaded PDF is the detailed endpoint/ownership source used here.	Update the living Project Context when the next document revision is made; no behavioral conflict was identified.

2. Architectural contract

2.1 Service ownership

Service	Framework and owned domain	Public contract responsibility
Gateway	Traefik	TLS, routing, JWT verification, CORS, rate limiting, request/correlation identifiers. Owns no business data.
Core	Django REST Framework	Identity, organizations, memberships, invitations, tenant registry, incidents, postmortem lifecycle and versions, comments, approvals, action items, analytics, exports, audit log, and job status.
Ingestion	Django REST Framework	Integrations, allowlists, service catalog, discovery, normalized evidence, timelines, and PagerDuty webhooks.
AI	Django Ninja and Celery	Model settings, draft runs, regeneration, blameless checking, inline AI suggestions, and usage limits.
Notifications	Django REST Framework and Celery	In-app notifications, notification settings, reminder and email delivery records.
Realtime	Later phase	Live document synchronization only; it is not a v1 API dependency.

Established source decision: services do not access another service's schema or import another service's implementation. Cross-service behavior uses synchronous internal REST where an answer is needed now, and durable events where another service reacts asynchronously.

2.2 Contract ownership and repository layout

Confirmed: contracts are centrally managed in the monorepo. Each service owns its implementation; shared API and event definitions live under a central `contracts/` directory, separated by service. Producers cannot silently change consumer-used schemas.

```
contracts/
  common/                # error, pagination, jobs, security headers, envelope
  core/                  # OpenAPI fragments and internal API schemas
  ingestion/
  ai/
  notifications/
  events/                # versioned event payload schemas by producer
```

Standard convention: maintain one machine-readable OpenAPI 3.x root document per public service plus reusable common components. DRF services publish with drf-spectacular; AI publishes the Django Ninja schema. CI validates both against the corresponding centrally managed contract fragments.

3. Public API rules

3.1 URLs, media type, identifiers, and verbs

Confirmed: public REST endpoints are versioned under `/api/v1`; use JSON request and response bodies; use ISO 8601 UTC timestamps; use UUIDv7 identifiers; and return direct resource bodies, adding metadata only when it is needed.

Operation	Convention
Read a collection/resource	`GET`
Create a resource or command-like child resource	`POST`
Replace an explicitly replaceable resource	`PUT`
Partially update a resource or state field	`PATCH`
Delete a resource	`DELETE`; apply the deletion policy in section 12
Start slow work	`POST`, return `202 Accepted` and a job resource

Standard convention: request and response content type is `application/json` except an SSE endpoint, which uses `text/event-stream`. `PATCH` is partial update; `PUT` is full replacement only where the inventory explicitly establishes it.

3.2 Tenant context and route scope

Confirmed: a user may belong to multiple organizations. The active organization is selected when Core issues the access token, and every tenant request must be bound to that active organization and the route's `{org_id}`. A cross-tenant object lookup returns `404`, not a tenancy-disclosing response.

For tenant resources the canonical public path prefix is:

```
/api/v1/orgs/{org_id}/...
```

Authentication and organization creation remain the established exceptions: they are versioned but not nested under an existing organization. The detailed auth paths are in section 5.1.

3.3 Resource metadata

Confirmed: return resources directly. Collections and operations add metadata only when navigation, concurrency, asynchronous state, or a server-generated result requires it.

Collection shape:

```
{
  "data": [ { "id": "uuidv7", "...": "resource fields" } ],
  "page": { "next_cursor": "opaque-or-null", "previous_cursor": "opaque-or-null" }
}
```

This is a standard convention for collections; an individual resource is the resource object itself, not a `{ "data": ... }` wrapper.

3.4 Error schema

Established source decision: every service uses one error shape: code, message, and optional field details.

```
{
  "code": "stale_version",
  "message": "The postmortem changed before this save.",
  "details": {
    "current_revision": 12
  },
  "correlation_id": "uuidv7-or-request-id"
}
```

`details` is optional and must contain only safe, actionable values. It must never expose secrets, internal stack traces, cross-tenant identifiers, or provider credentials.

Status	Contract use
`400`	malformed request or unsupported query parameter
`401`	absent, expired, or invalid authentication
`403`	authenticated but not permitted, where revealing existence is appropriate
`404`	absent resource, including cross-tenant resource lookup
`409`	state-transition conflict or other non-ETag conflict
`412`	failed `If-Match` optimistic-concurrency precondition
`422`	syntactically valid body that fails field/domain validation
`429`	scoped rate limit; includes `Retry-After`
`5xx`	service failure; response remains user-safe

3.5 Filtering, sorting, and pagination

Confirmed: timelines and logs use cursor pagination; ordinary lists use page pagination. Cursor navigation must be stable. Filtering and sorting follow one documented query convention.

Collection type	Required query contract
Timeline and audit/activity logs	`cursor`, `limit`, `sort`; cursor captures the complete ordered position so inserts before/after a viewed page do not duplicate or skip records while navigating.
Ordinary lists such as incidents, action items, services, notifications, and postmortem library	`page`, `page_size`, declared filters, and `sort`.

Standard convention: filters are named from public resource fields (for example, `status`, `severity`, `service_id`, `owner_user_id`, `created_at_gte`, `created_at_lte`). `sort` is a comma-separated list; `-field` is descending and `field` is ascending. Each endpoint declares its allowed filters and sort fields in OpenAPI; unlisted fields are rejected with `400`.

Open implementation item: exact page-size defaults and maximums are configurable limits and must be declared per endpoint before release. Do not invent a numeric default.

3.6 Idempotency and concurrency

Confirmed: create/command requests accept `Idempotency-Key`. The server retains a completed request key and its outcome for **36 hours**, scoped so it cannot replay across tenant, actor, or operation boundaries. Reuse with the same operation and same canonical request returns the original outcome. Reuse with a different request returns a conflict error.

Confirmed: mutable important resources use optimistic concurrency with an `ETag` response header and an `If-Match` request header. A failed match returns `412` with `stale_version`. ETags represent the server's current version of that resource; clients must not construct them.

The postmortem working draft additionally retains its established `revision` behavior. The API may expose it for editor recovery, but `ETag`/`If-Match` is the HTTP concurrency contract.

3.7 Rate limits

Confirmed: rate limits are configurable and scoped; a rate-limited response is `429` with `Retry-After`.

Standard convention: scopes may distinguish unauthenticated auth, user, organization, service identity, integration/provider, endpoint, and expensive AI operation. Numeric limits, windows, and provider-specific backoff remain configuration, not this contract.

4. Endpoint inventory

This is the inventory established by the System Design. All tenant paths below are prefixed with `/api/v1/orgs/{org_id}` unless explicitly marked otherwise. It is an inventory, not permission to invent undocumented subroutes or payload fields.

4.1 Core

Area	Endpoint(s)	Ownership and contract note
Auth	`POST /api/v1/auth/register`, `/login`, `/logout`, `/refresh`, `/verify-email`, `/password-reset`; `GET /api/v1/auth/oauth/{provider}/start`, `/callback`; `GET /api/v1/me`	Core. Public until authentication requires a user. Exact verification/reset body fields are open.
Organizations	`POST /api/v1/orgs`; `GET`, `PATCH /api/v1/orgs/{org_id}`; `GET`, `PATCH /api/v1/orgs/{org_id}/settings`	Core. Members read; Admin writes settings.
Members and invitations	`GET /members`; `PATCH`, `DELETE /members/{id}`; `POST`, `DELETE /invitations`; `POST /invitations/accept`	Core. Admin operations, subject to the established at-least-one-active-Admin rule.
Incidents	`GET`, `POST /incidents`; `GET`, `PATCH /incidents/{id}`; `POST /incidents/{id}/description`	Core. Members write; all members read.
Postmortems	`GET /incidents/{id}/postmortem`; `POST /incidents/{id}/postmortem/generate`; `PUT /postmortems/{id}`; `GET /postmortems/{id}/versions`; `POST /postmortems/{id}/submit`, `/approve`, `/request-changes`, `/publish`, `/reopen`	Core lifecycle owner. Generation is asynchronous. `PUT` is the established autosave/working-draft endpoint.
Comments	`GET`, `POST /postmortems/{id}/comments`; `PATCH /comments/{id}`	Core. All members under role rules.
Action items	`GET`, `POST /action-items`; `PATCH /action-items/{id}`; `POST /action-items/{id}/verify`	Core.
Analytics	`GET /analytics/recovery-times`, `/analytics/incident-volume`, `/analytics/action-health`	Core.
Exports	`POST /postmortems/{id}/export`; `GET /exports/{id}/download`	Core. Export creation is asynchronous; download behavior remains signed-link based.
Audit	`GET /audit-log`; `GET /audit-log/verify`	Core. Admin. Audit records remain append-only.
Jobs	`GET /jobs/{job_id}`	Core public job-status surface. Requester or Admin only.

4.2 Ingestion

Area	Endpoint(s)	Ownership and contract note
Integrations	`GET /integrations`; `POST /integrations/{provider}/connect`; `GET /integrations/{provider}/callback`; `DELETE /integrations/{id}`; `GET`, `PUT /integrations/{id}/allowlist`	Ingestion. Admin. Secrets are write-only and responses show only a masked hint where relevant.
Service catalog	`GET`, `POST`, `PATCH`, `DELETE /services`; `PUT /services/{id}/links`	Ingestion. Admin writes; members read.
Evidence	`GET /incidents/{id}/evidence`; `PATCH /evidence/{id}`; `POST /incidents/{id}/evidence`; `POST /incidents/{id}/evidence/confirm`	Ingestion. `PATCH` includes/excludes. Adding accepts a link or text; outbound fetching is limited to connected Slack, GitHub, and PagerDuty hosts, otherwise it is stored as a note without fetching.
Timeline	`GET /incidents/{id}/timeline`; `POST`, `PATCH`, `DELETE /timeline-events/{id}`	Ingestion. Timelines use cursor pagination.
Webhook	`POST /webhooks/pagerduty/{token}`	Ingestion. Provider signature verification and old-timestamp rejection are required. This route is versioning/open-gateway placement work; its exact public prefix must be recorded in the OpenAPI gateway configuration.

4.3 AI

Area	Endpoint(s)	Ownership and contract note
Model setting	`GET`, `PUT /ai-model`; `POST /ai-model/test`	AI. Admin. Provider, model identity, endpoint, and key rules remain the AI/model-setting contract; no provider behavior is invented here.
Section regeneration	`POST /postmortems/{id}/sections/{key}/regenerate`	AI. Members. Creates a suggestion/new immutable postmortem version through Core; it never overwrites human edits.
Blameless check	`POST /postmortems/{id}/blameless-check`; `GET /postmortems/{id}/blameless-suggestions`; `PATCH /blameless-suggestions/{id}`	AI. Members. Finding actions are accept, edit, or dismiss.
Inline AI	`POST /postmortems/{id}/ai/edit`	AI. Members. Uses SSE for streamed suggestions; suggestions are not saved until accepted.

4.4 Notifications

Area	Endpoint(s)	Ownership and contract note
Notifications	`GET /notifications`; `PATCH /notifications/{id}`	Notifications. Patch marks read.
Preferences	`GET`, `PATCH /notification-settings`	Notifications.

4.5 Internal APIs

Established source decision: internal endpoints are service-token only and reachable only on the internal network. Core exposes tenant resolution, postmortem text retrieval, generated-draft version creation, stalled/nearly-due action-item listing, and user display-name lookup. Ingestion exposes incident evidence and timeline retrieval to AI. Exact internal route names, request fields, and service-token claims are **open implementation items**; they must be defined in each service's central contract folder before a caller is built.

5. Authentication, authorization, and tenant security

5.1 Session contract

Confirmed / established: Core issues short-lived signed JWT access tokens and DB-backed rotating refresh tokens. Access tokens carry the user, active organization, and role. Refresh tokens are stored as hashes, rotate on use, can be revoked, and reuse of a rotated token revokes its token family. The browser receives the refresh token in an HttpOnly, Secure cookie on the shared parent domain.

Item	Contract
Access token	Bearer JWT, signed with Core's asymmetric signing key; every gateway/service verifies published public keys with rotation support. Exact lifetime remains the established short-lived 10-15 minute configuration range, not a new hard-coded API value.
Refresh token	Opaque credential, database-backed and hash-stored; cookie is never returned in JSON. `POST /auth/refresh` rotates it.
Logout	Revokes the relevant refresh token/family according to Core's session policy.
Active organization	Chosen through Core token issuance; must agree with `{org_id}` on a tenant route. Switching organization requires a token for the selected organization.
Service identity	Managed identity or client credentials produces a service token; only known services are accepted on the internal network.

5.2 Layered authorization

Confirmed: authorization is layered. The gateway validates token integrity; each service validates audience/identity and binds tenant context; endpoint permissions enforce role and operation; domain guards enforce record-state constraints; the database role enforces service-schema isolation. UI hiding is cosmetic only.

Role	Established v1 capability summary
Admin	Membership, integrations, allowlists, model configuration, audit visibility, approval/publish, and all Member capabilities.
Member	Create incidents/generations; edit and comment within lifecycle limits; create/update action items; view permitted tenant resources.
Manager	Optional/later role; may manage catalog and approve when enabled.
Viewer	Optional/later role; read published material and analytics, comment-only postmortem access where enabled.

Critical actions including role changes, approval, and deletion must re-check current authorization through Core instead of relying only on an unexpired JWT.

5.3 Security rules

- Tenant data must never be selected, cached, or emitted without a resolved tenant context.
- Integration and model secrets are write-only API values. Responses expose a masked hint only where the owning service already supports it.
- Evidence is untrusted data. Redaction and allowlist checks happen before storage and before model use.
- Webhooks verify provider signature, reject stale timestamps, and are idempotent.
- Audit entries are append-only and hash chained; audit verification is read-only.
- New external behavior, provider semantics, scopes, and credential exchange fields remain source- or provider-specific work, not generic contract assumptions.

6. Request and response contracts

The following logical schemas are implementation constraints, not permission to add undocumented database columns. Required/optional field-level OpenAPI definitions belong in central contract files and must be mapped only to established persistence and service fields.

6.1 Asynchronous command accepted response

Confirmed: Celery-backed slow work returns `202 Accepted` with a `job_id`.

```
{
  "job_id": "018f0d8b-0000-7000-8000-000000000000",
  "status": "queued"
}
```

`job_id` is a UUIDv7. The established job states are `queued`, `running`, `succeeded`, `failed`, and `cancelled`; AI draft runs may additionally expose their established `partial` terminal outcome. The canonical public job schema must declare how a cross-service job maps these states before UI implementation.

6.2 Job status and progress

```
{
  "id": "uuidv7",
  "kind": "generate_draft",
  "status": "running",
  "progress": 0,
  "entity_type": "incident",
  "entity_id": "uuidv7",
  "error": null,
  "correlation_id": "uuidv7"
}
```

The known job storage model includes kind, status, requester where available, entity reference, progress, timestamps, and a user-safe error message. Exact progress stage names and percent semantics are **open**; clients must not infer hidden AI stages from an integer.

6.3 Direct-resource examples

`GET /incidents/{id}` returns the incident directly, using established incident fields such as `id`, `source`, `external_id` when applicable, `title`, `description`, `incident_type`, `severity`, `status`, known lifecycle times, affected services, and responders. The source defines these fields; field nullability and editability belong in Core OpenAPI.

`GET /incidents/{id}/evidence` returns a paged direct list of the Ingestion-owned evidence representation. Each normalized evidence item must expose section 10's timestamp contract in addition to established evidence fields: source, source reference, original URL when available, actor/display label when available, kind, redacted text, inclusion state, and source-specific metadata allowed by the contract.

`GET /incidents/{id}/timeline` returns cursor-paginated timeline events. Timeline items retain their stable event identifiers, summary/detail, source, key moment/phase when present, discrepancy indication, manual/hidden state where applicable, and evidence references as established by Ingestion.

`PUT /postmortems/{id}` is the working-draft autosave route. It requires `If-Match`, accepts the ProseMirror document and the established base revision behavior, and returns the updated direct postmortem resource with a new `ETag`. It does not create a named version merely because it autosaves.

6.4 Postmortem version and generation contract

Confirmed: named postmortem versions are immutable. Explicit save and AI generation/regeneration create a version; autosave updates only the temporary working draft. The active named version queue contains the most recent 15 versions. AI output is always a draft or suggestion; human edits are never overwritten.

A named version must retain the established provenance when applicable: trigger, label, snapshot content, creator snapshot, creation time, generated draft-run reference, model/prompt version, and blameless state. The AI Pipeline Specification further requires evidence snapshot provenance for generated output; the exact storage mapping is open and must be reconciled with the Database Design.

6.5 AI inline SSE contract

Confirmed: progress for background jobs is available through SSE with polling fallback; inline AI streams suggestions with SSE. The stream must be authenticated and tenant-authorized before it begins. SSE is delivery/progress transport, not a second state-mutating API.

Standard convention: use named SSE events `progress`, `result`, `error`, and `complete`. Each data payload includes a `correlation_id`; `progress` includes `job_id` when a durable job exists. The exact endpoint path, reconnect behavior, and payload fields are open and must be published in AI OpenAPI before frontend implementation. Polling `GET /jobs/{job_id}` remains the universal fallback.

7. Eventing contract

7.1 Transport and delivery

Established source decision: Redis Streams carries cross-service events behind an event-publisher interface. Each producer writes an outbox row in the same transaction as the domain change. A relay publishes after commit. Consumers use a service-specific consumer group and an inbox/idempotency record.

Confirmed: delivery is at least once. Consumers must be idempotent, deduplicate by `event_id` plus consumer identity, and safely handle stale or out-of-order aggregate versions.

7.2 Standard event envelope

Confirmed: every event has this envelope. Fields are required unless the value is legitimately absent, in which case the event is not ready to publish rather than substituting another identifier.

```
{
  "event_id": "uuidv7",
  "event_type": "postmortem.approved",
  "event_version": 1,
  "occurred_at": "2026-11-06T00:00:00Z",
  "tenant_id": "uuidv7",
  "aggregate_type": "postmortem",
  "aggregate_id": "uuidv7",
  "aggregate_version": 1,
  "producer": "core",
  "correlation_id": "uuidv7",
  "payload": {}
}
```

Field	Contract
`event_id`	UUIDv7 generated once by the producer; immutable and unique.
`event_type`	Lowercase dot-separated domain name.
`event_version`	Positive schema version for this event type; evolves independently of `/api/v1`.
`occurred_at`	UTC time the domain event occurred, not the relay publish time.
`tenant_id`	Organization/tenant UUID. Every tenant event is tenant-scoped.
`aggregate_type`	Domain aggregate name, not a database table name.
`aggregate_id`	Stable aggregate UUID.
`aggregate_version`	Monotonically increasing aggregate state version used for stale/out-of-order handling.
`producer`	Producing service identifier: `core`, `ingestion`, `ai`, or `notifications`.
`correlation_id`	Request/workflow correlation identifier propagated across commands, jobs, and events.
`payload`	Small, schema-versioned facts and identifiers only; never evidence bodies, document content, raw provider payloads, secrets, or credentials. Consumers fetch detail through authorized APIs.

Open implementation item: exact aggregate-version persistence/increment strategy must be chosen per aggregate and added to its owning service schema. The existing older outbox description lacks `event_id`, `event_version`, tenant, producer, correlation, and aggregate version fields; this is required migration work, not a reason to weaken the envelope.

7.3 Consumer ordering, retries, and replay

1. A consumer records receipt by `event_id` and consumer name before or atomically with its effect where feasible.
2. If it has already completed the event, it returns success without repeating the business effect.
3. For a known aggregate, a lower `aggregate_version` than the consumer's applied version is stale and is ignored after recording a safe outcome.
4. A higher aggregate version that creates a detectable gap is not applied blindly. The consumer fetches current state from the producer's authorized API or records a retryable ordering gap. It must not manufacture intermediate state.
5. Failed delivery uses bounded retry with backoff, then moves to a dead-letter stream. Retry counts, delay curve, and dead-letter retention are configurable and not numerically fixed here.
6. Controlled replay is an explicit operator/admin workflow with an audit record, a bounded event selection, and correlation/replay metadata. A replay never disables consumer idempotency and must not create a second business effect for an already completed event.

7.4 Event catalogue

The following names are confirmed from the System Design, normalized to the confirmed lowercase dot naming convention.

Event type	Producer	Consumers	Aggregate	Purpose and minimum payload
`incident.reported`	Ingestion	Core	incident	PagerDuty incident created or resolved. Payload: the provider incident reference needed for Core to create/update the incident. Exact fields open.
`incident.description_submitted`	Core	Ingestion	incident	Author description became available and discovery should start. Payload: incident identifier only plus small discovery-relevant identifiers approved by Ingestion contract.
`evidence.ready`	Ingestion	Core	incident	Candidate evidence is ready for preview. Payload: incident id and discovery/timeline revision reference; exact payload open.
`timeline.ready`	Ingestion	Core	incident	Confirmed timeline was built. Payload: incident id and timeline revision/reference.
`draft.requested`	Core	AI	postmortem or incident	Generate a draft against the authorized evidence snapshot. Payload: incident id, postmortem id where present, requester id, and snapshot reference; exact wire fields open.
`draft.ready`	AI	Core, Notifications	draft run	AI produced a structured result. Core persists a new version; Notifications may notify. Payload: draft run id, incident/postmortem references, resulting status, and safe provenance references.
`postmortem.submitted`	Core	Notifications	postmortem	Postmortem entered review. Payload: postmortem id and submitted version reference.
`postmortem.approved`	Core	Notifications	postmortem	A permitted non-author approved a specific version. Payload: postmortem id and approved version reference.
`tenant.provisioned`	Core	All services	tenant	Tenant is ready for use. Payload: tenant id only unless service initialization requires another approved identifier.
`audit.recorded`	Every service	Core	audit record / source aggregate	An auditable action must be appended to Core's hash-chained audit log. Payload must be safe audit facts only. Exact event aggregate convention is open.
`incident.deleted`	Core	Ingestion, AI, Notifications	incident	Required by the Database Design to clean related cross-schema records. Payload: incident id.

Open implementation items: no additional event names, payload fields, or provider-specific events are implied. The exact event schemas, including `aggregate_type`, payload requirement, version, and aggregate-version source, must be committed per row in `contracts/events/` before a producer or consumer ships.

8. Background jobs and lifecycle

Confirmed: Celery runs service-owned slow work only. Jobs are never shared between services. Public command requests return `202` and a `job_id`; cross-service work uses events or REST, not a shared Celery task.

Service	Established jobs
Core	`provision_tenant`, `render_export`, `refresh_analytics_summaries`, `verify_audit_chain`, `purge_expired_data`, `expire_invitations`, `outbox_relay`
Ingestion	`discover_evidence`, `build_timeline`, `sync_pagerduty_incidents`, `refresh_integration_health`, `outbox_relay`
AI	`generate_draft`, `regenerate_section`, `run_blameless_check`, `outbox_relay`
Notifications	`scan_stalled_items`, `send_reminder`, `send_email`, `outbox_relay`

8.1 Lifecycle rules

- A job is created before or atomically with the user-visible asynchronous command outcome according to its service's transaction boundary.
- `GET /jobs/{job_id}` is authorization-checked and is the polling fallback for all progress UX.
- SSE reflects progress; it does not replace the job record as the authoritative state.
- Cancellation is only exposed where the service can safely stop work. Evidence exclusion has the explicit stale/cancel behavior in section 10.3; no generic cancellation endpoint is invented.
- User-safe failures are visible to callers; detailed diagnostics remain in service logs/observability.

9. Evidence, timeline, and timestamp contracts

9.1 Evidence scope and provenance

Evidence belongs to Ingestion. It is obtained only from organization-authorized allowlists and accepted manual input, is normalized and redacted, and is associated with an incident. Raw provider payloads are not retained as the normal evidence representation. Source-specific authority applies where appropriate, but contradictory evidence is preserved rather than silently merged or discarded.

Every AI run receives only authorized, included, redacted evidence from its exact immutable evidence snapshot. Stable source/timeline event identifiers persist through preparation, claims, citations, editor display, and exports.

9.2 Five timestamp fields

Confirmed: every normalized evidence event exposes these five fields.

Field	Meaning
`occurred_at`	Normalized occurrence time in UTC only when it can be represented without inventing certainty.
`occurred_at_raw`	Original provider timestamp representation.
`observed_at`	Time Retrace retrieved or recorded the event.
`timestamp_precision`	Precision actually supplied by the source, such as exact, second, minute, date-only, or unknown. The exact enumeration is an open schema item.
`timezone_known`	Whether the source timestamp's timezone is known.

`occurred_at` must never imply more precision or timezone certainty than the source supplied. A date-only or timezone-ambiguous source must not become an invented exact UTC instant. Timeline ordering must surface material timestamp discrepancies as established by the product and system design.

9.3 Evidence inclusion/exclusion and snapshots

Confirmed: include/exclude changes create a new immutable evidence snapshot. Any running AI job against the previous snapshot becomes stale or is safely cancelled. New generation uses only the current snapshot. Existing postmortem versions remain immutable and retain their original snapshot reference.

This applies to `PATCH /evidence/{id}` and any other established evidence change that affects inclusion, content, or authorized evidence scope. It does not delete historical postmortem content merely because a newer snapshot excludes the evidence; access/export behavior for previously retained versions follows the product's retention and deletion policy.

Open implementation item: the snapshot resource/path, exact status vocabulary for stale vs cancelled, snapshot-hash fields, and storage mapping must be defined jointly by Core, Ingestion, and AI. The behavior is confirmed; the wire detail is not.

10. Postmortem lifecycle, deletion, and concurrency

10.1 State transitions

The established postmortem states are `draft`, `in_review`, `changes_requested`, `approved`, and `published`.

Command	Guard
Submit	Latest version has a completed blameless check and no generation is running.
Request changes	Admin; a note is required.
Approve	Admin (or Manager when the optional role exists), not the author; reviewed version must be latest.
Publish	Admin; approval requirement applies.
Reopen	Authorized user; store a snapshot first, return to review, and lock/unlock behavior follows established lifecycle.

10.2 Version behavior

- Named versions are append-only and immutable.
- Explicit save and AI regeneration create named versions.
- Autosave writes only the current working document and must use `If-Match` to prevent silent overwrites.

- A regeneration result is a suggestion/new version, never an implicit destructive edit of human text.
- Approved/published documents are read-only until reopened.
- Claim citations remain resolvable to stable timeline events in editor, PDF, and Markdown output.

10.3 Deletion

Confirmed: important records use soft deletion by default at the public API boundary. A deleted resource is excluded from ordinary lists and reads unless an explicitly authorized restore/audit workflow is defined. The exact response to successful soft delete (`204` or a resource body) and resource-by-resource restore endpoints are open; do not invent them.

Established exceptions: deleting an incident removes its evidence, drafts, and comments; deleting an organization stops access and performs the documented tenant-database, role, key, secret, Blob, and registry cleanup. These destructive flows are not converted into a no-op soft delete by this default.

10.4 Concurrency

Every mutable important resource returns `ETag`; update/delete commands require `If-Match`. A stale postmortem save returns `412`, leaving both the stored working draft and immutable versions unchanged. State-machine conflicts that are not stale representations return `409` with a specific code.

11. OpenAPI, compatibility, and test gates

11.1 Contract publication

Confirmed: OpenAPI 3.x is generated with drf-spectacular for DRF services and Django Ninja's schema support for AI. All public and internal HTTP contracts are represented in the central monorepo contracts, with implementation schemas validated against them.

Standard convention: publish a stable `/api/v1` OpenAPI artifact during CI/build. Generated typed clients may be used by the frontend, but generated code does not replace contract review.

11.2 Compatibility and versioning

Surface	Rule
HTTP API	Non-breaking additions may remain in `/api/v1`. Remove/rename/change meaning only through a deliberate new API version and migration/deprecation plan.
Event type	Use `event_version` for compatible schema evolution. Consumers must tolerate fields they do not use; producers must not remove/alter required meaning without coordinated versioning.
Event name	Lowercase dot names are stable domain identifiers. A materially new meaning uses a new event type or deliberate version plan, not an undocumented semantic change.
Data contract	Database migrations are owned by the service and must support the release/rollback plan. No cross-schema table access is introduced to avoid a contract change.

11.3 CI requirements

Confirmed: contract-first tests run in CI.

Minimum gates:

1. Generate each service's OpenAPI document and diff/validate it against central contract definitions.
2. Validate request/response examples and error envelopes.
3. Validate event envelope and each event payload schema by version.
4. Run producer/consumer compatibility fixtures, including duplicated, stale, and out-of-order events.
5. Test 36-hour idempotency-key replay behavior with same and mismatched request bodies.
6. Test `ETag`/`If-Match` stale-write behavior.
7. Test cursor stability, page-list filtering/sorting, and rate-limit `429`/`Retry-After` shape.
8. Test tenant isolation across routes, jobs, caches, event consumers, and direct database roles.

9. Test evidence exclusion snapshot invalidation, stale/cancel behavior, and preservation of prior version snapshot references.
10. Test AI's evidence-only, citation, contradiction, timestamp-uncertainty, and no-human-overwrite invariants using the AI Pipeline Specification's matrix.

12. Confirmed decisions index

The following checklist records the 30 settled API/event decisions for implementation traceability.

1. REST synchronous APIs plus asynchronous jobs/events.
2. `/api/v1` URL versioning.
3. Short-lived JWT access tokens plus DB-backed rotating refresh tokens.
4. Multi-organization membership and active-organization context.
5. Standard event envelope with the eleven specified fields.
6. At-least-once events with idempotent consumers.
7. Aggregate-version stale/out-of-order handling.
8. Retry/backoff/dead-letter with controlled replay.
9. `Idempotency-Key` retained for 36 hours.
10. Cursor pagination for timelines/logs and page pagination for ordinary lists.
11. Stable cursor navigation.
12. Standard documented query filtering and sorting.
13. Direct resources, with metadata only where needed.
14. Conventional REST verbs.
15. Soft delete important records by default, subject to stated destructive exceptions.
16. `ETag`/`If-Match` optimistic concurrency.
17. Immutable rolling 15-version postmortem queue.
18. Explicit save and AI regeneration create versions; autosave is temporary working draft.
19. Configurable scoped rate limits with `429` and `Retry-After`.
20. Celery jobs return `202` plus `job_id`; SSE progress with polling fallback.
21. Layered authorization.
22. OpenAPI 3.x through drf-spectacular and Django Ninja schema.
23. Contract-first testing in CI.
24. Lowercase dot domain event names.
25. Five explicit evidence timestamp fields.
26. Evidence exclusion creates snapshot, stales/cancels old jobs, and preserves prior version references.
27. Centrally managed monorepo contracts, one service folder per service.
28. Source-specific authority where appropriate, while preserving contradictions.
29. Evidence-only AI reasoning, immutable provenance, and human-controlled root-cause confirmation.
30. No invented provider behavior, unresolved numeric limits, route details, columns, or service ownership; unresolved details are explicit contract work.

13. Open implementation register

ID	Item	Owner(s)	Blocking scope
API-01	Complete exact OpenAPI request/response fields, validation, and filters for every inventory endpoint.	Owning service	Endpoint implementation and generated client
API-02	Define webhooks' versioned gateway path and OpenAPI exposure without changing signature-verification behavior.	Gateway, Ingestion	Webhook deployment
API-03	Define public job resource mapping and SSE endpoint/reconnect payload details.	Core, AI, frontend	Job-progress UI
EVT-01	Commit versioned payload schemas, aggregate types, and aggregate-version sources for every catalogue event.	Producers and consumers	Event producer/consumer implementation
EVT-02	Migrate outbox/inbox data model to support the confirmed envelope and replay audit metadata.	Each service	Reliable eventing
EVD-01	Define evidence-snapshot wire resource, storage mapping, hash/reference, and stale/cancel statuses.	Core, Ingestion, AI	Evidence changes and AI generation
DB-01	Map five evidence timestamp fields and soft-delete lifecycle to approved migrations without inventing columns in API code.	Ingestion, Core	Evidence/deletion implementation
PM-01	Define precise rolling-queue retention/archive mechanics and explicit-save command surface.	Core	Version lifecycle
SEC-01	Define service-token claims/audience and internal endpoint schemas.	Core and all services	Internal calls
CFG-01	Set configurable numeric page sizes, rate limits, retries, backoff, dead-letter retention, and progress-stage behavior through configuration/evaluation.	Service owners	Production configuration

14. Coding-agent implementation checklist

1. Read this document with the PRD, System Design, Database Design, Project Context, and AI Pipeline Specification before changing contracts.
2. Work only in the owning service and central `contracts/` folder; never read another service's tables directly.
3. Add or update the OpenAPI/event schema first, then implementation, then contract fixtures.
4. Use `/api/v1` and the tenant-prefix rule exactly; do not add routes from intuition.
5. Bind every tenant request/job/event to the active organization and reject cross-tenant access as `404`.
6. Implement `Idempotency-Key` retention for 36 hours and `ETag`/`If-Match` before adding mutating UI flows.
7. Return `202` plus `job_id` for slow commands; expose authorized job polling and SSE with fallback.
8. Use the standard event envelope, transactional outbox, inbox deduplication, aggregate-version checks, retries, dead-letter handling, and audited replay.
9. Keep event payloads small; fetch full/evidence data through authorized APIs.
10. Preserve evidence uncertainty, source-specific authority, contradictions, and snapshot provenance. Never manufacture timestamps or source certainty.
11. On evidence exclusion, create a new snapshot and stale/cancel affected AI jobs; never rewrite previous postmortem versions.
12. Preserve the 15-version immutable queue, explicit-save/AI-version behavior, and temporary autosave model.
13. Treat soft deletion as a migration-backed contract change; honor incident/organization destructive-cleanup exceptions.
14. Do not pick a provider, a provider semantic, a model, a numeric limit, a missing column, an internal route, or an event payload field without resolving the corresponding open item.
15. Add contract-first CI coverage for normal, invalid, duplicate, stale, out-of-order, cross-tenant, and retry cases before shipping.

15. Source baseline

This contract was reconciled against the uploaded Retrace PRD v0.6, System Design PDF (labeled v0.3), Database Design v0.1, Project Context PDFs (identical copies dated 29 October 2026), and AI Pipeline Specification v0.2, plus the confirmed decisions captured in the referenced project conversation. Source decisions are preserved unless an explicit reconciliation is listed in section 1.3.